

CMPS 405: OPERATING SYSTEMS

Memory Management: Virtual Memory

This material is based on the operating system books by Silberschatz and Stallings and other Linux and system programming books.

Objectives

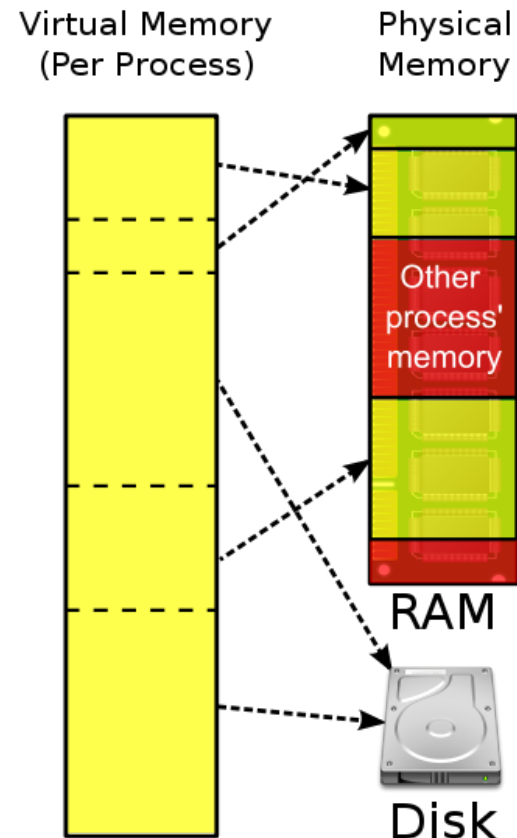
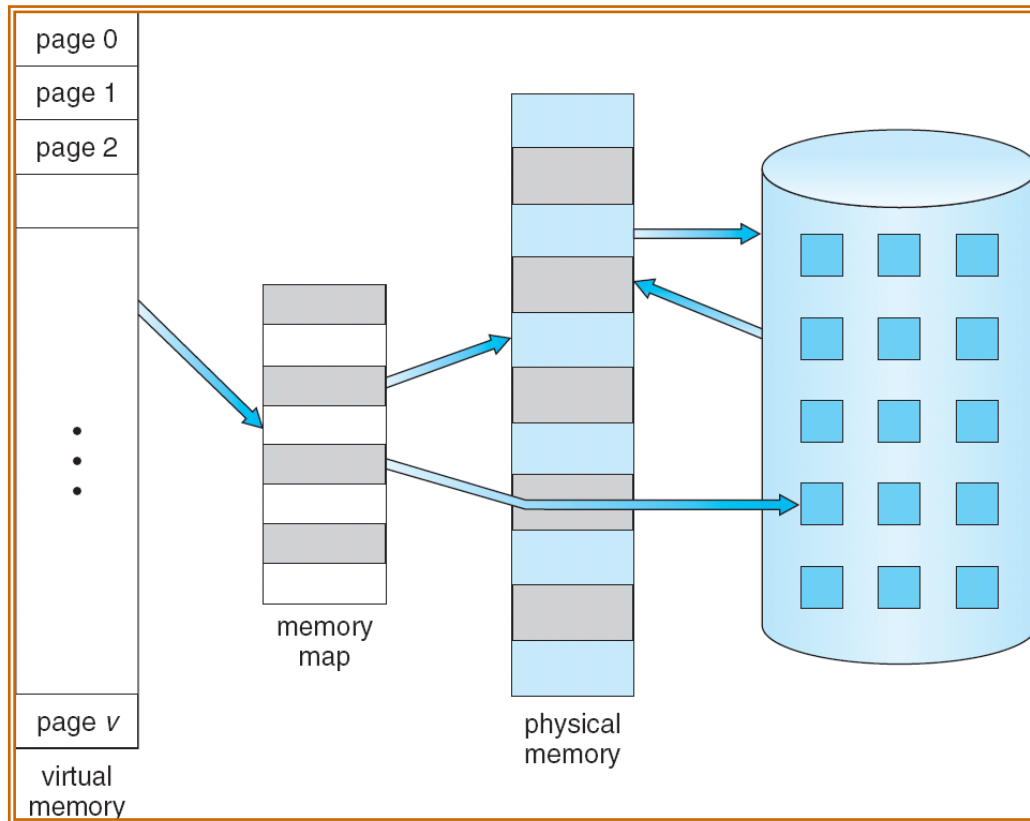
- ❖ To introduce the terminology of **virtual memory**.
- ❖ To explore different **virtual memory implementations**.
- ❖ To present a number of different **methods** for **page replacement**.

Topics

- ❖ Motivation
- ❖ Virtual Memory
 - Demand paging
 - Page fault handling
 - Page Replacement Algorithms
- ❖ Page Table
 - Structure
 - Implementations
- ❖ Additional topics
 - Thrashing and Prepaging
 - LRU Implementations and Approximations
- ❖ Allocating Kernel Memory
 - Buddy system

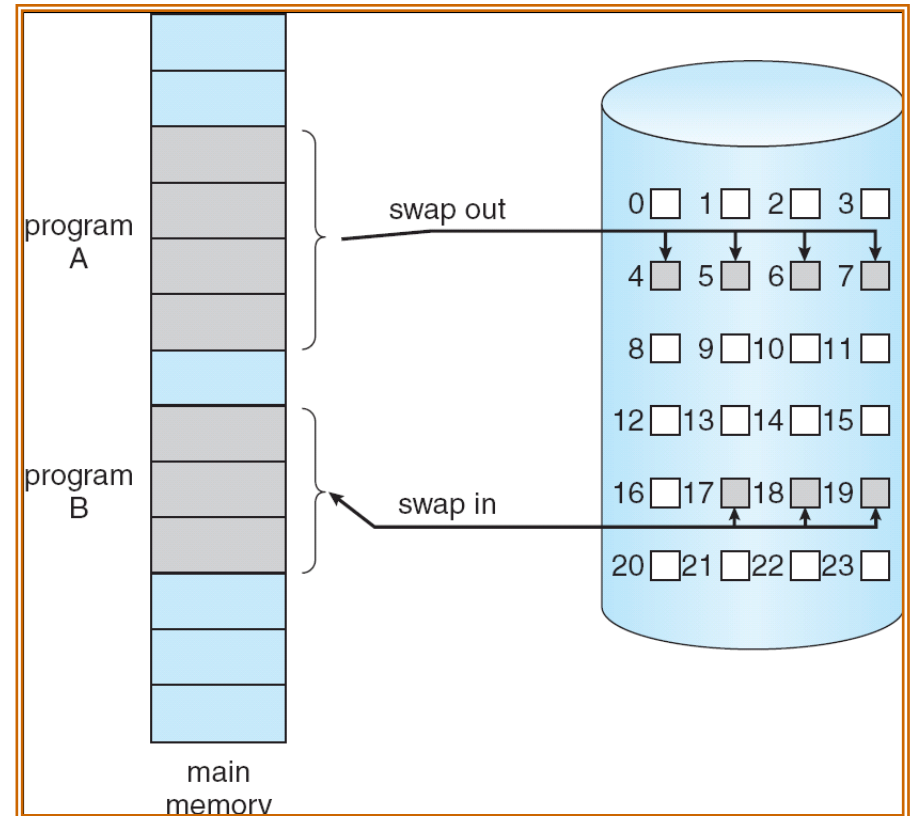
Virtual Memory

- ❖ The **program thinks** it has a **large range of contiguous addresses**; but in **reality** the **parts** it is **currently using** are **scattered** around **RAM**, and the **inactive parts** are saved in a **disk file**



Background

- ❖ Since only part of the program needs to be in memory for execution:
 - **Logical address space** can therefore be **much larger** than **physical address space**
 - Allows for **more efficient process creation**
- ❖ **Virtual memory implementations:**
 - Demand paging
 - Demand segmentation



Demand Paging

- ❖ **Bring a page into memory** only when it is **needed**
 - Less memory needed
 - Faster response
 - More users
- ❖ Page is needed $\Rightarrow$ reference to it
 - illegal memory reference (not in logical address space of the process) $\Rightarrow$ abort
 - not-in-memory (exists on disk only) $\Rightarrow$ bring to memory
- ❖ **Lazy swapper** – never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**

Valid-Invalid Bit

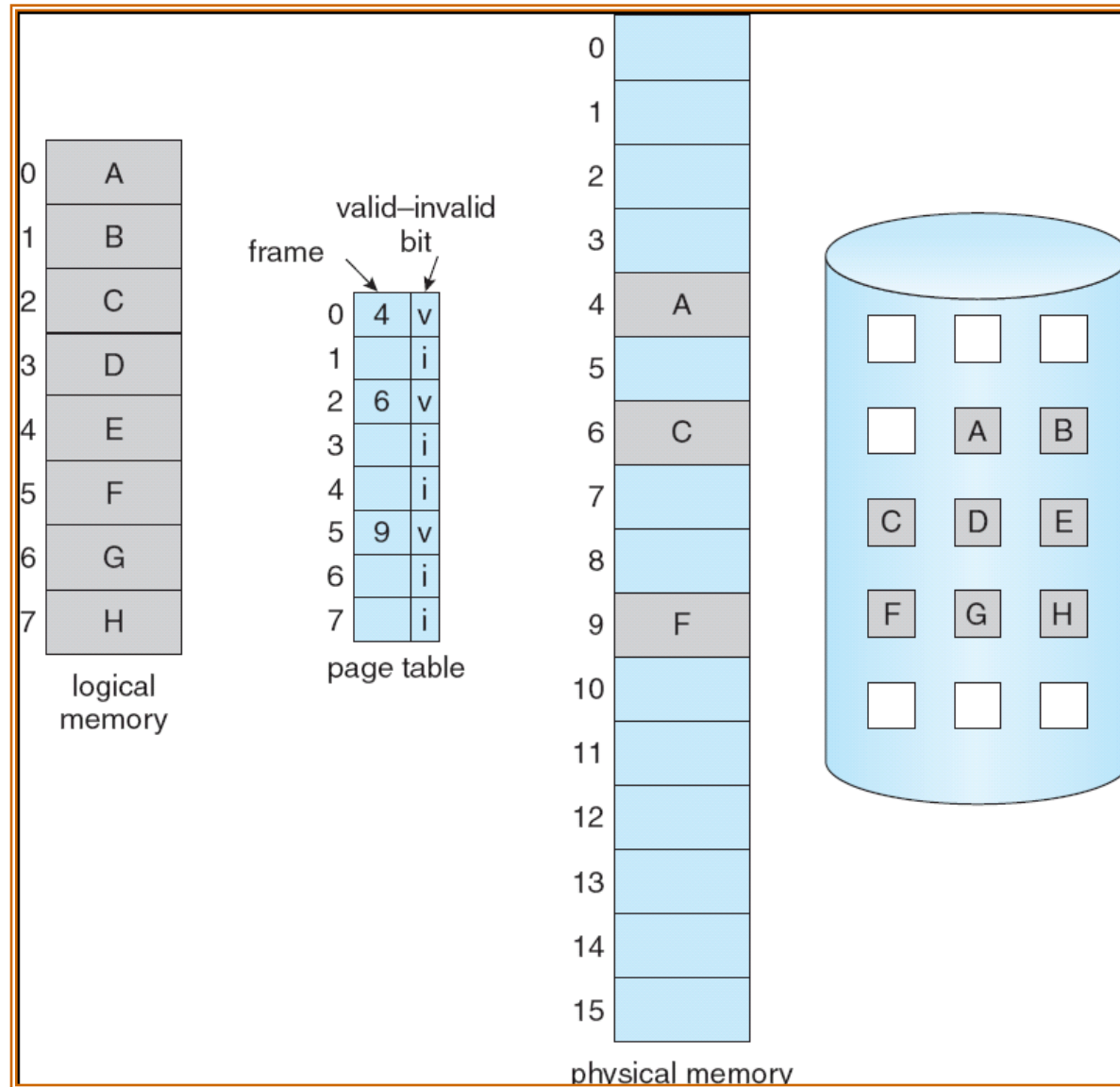
- ❖ With **each page table entry** a **valid–invalid bit** is associated
(**v** $\Rightarrow$ in-memory, **i** $\Rightarrow$ not-in-memory)
- ❖ **Initially valid–invalid bit** is set to **i** on all entries
- ❖ Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	v
	i
....	
	i
	i

page table

- ❖ During **address translation**, if **valid–invalid bit** in page table entry is **i** $\Rightarrow$ page fault

Page Table When Some Pages Are Not in Main Memory



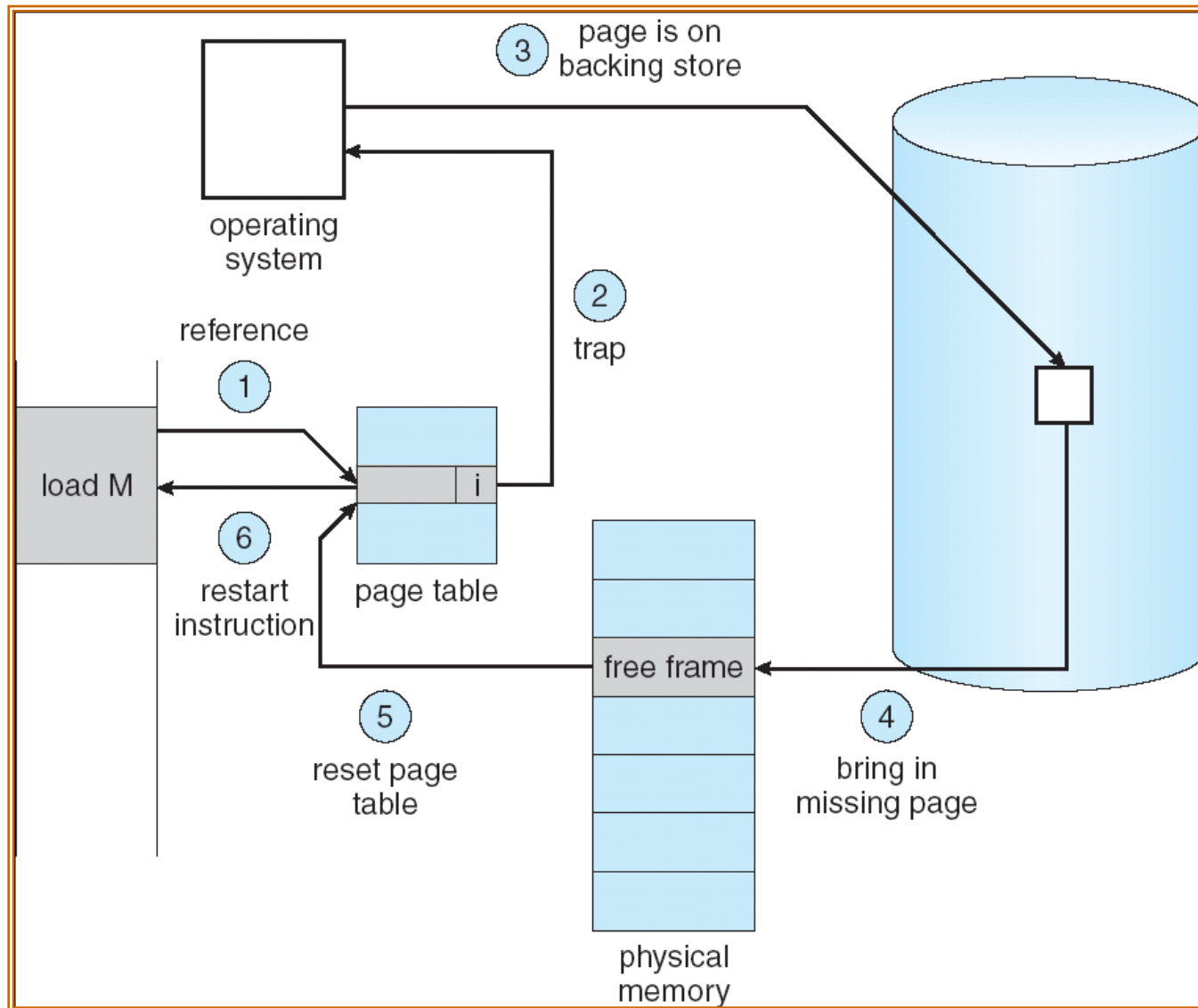
Page Fault

- ❖ If there is a **reference to a page**, **first reference** to that page will **trap** to operating system:

page fault

1. Operating system looks at another table to decide:
 - illegal memory reference $\Rightarrow$ abort
 - Just not in memory
2. Get empty frame
3. Swap page into frame
4. Reset tables
5. Set validation bit = **v**
6. Restart the instruction that caused the page fault

Steps in Handling a Page Fault



Performance of Demand Paging

❖ Page Fault Rate $0 \leq p \leq 1.0$

- if $p = 0$ no page faults
- if $p = 1$, every reference is a fault

❖ Effective Access Time (EAT)

$$\begin{aligned} \text{EAT} = & (1 - p) \times \text{memory access} \\ & + p (\text{page fault overhead} \\ & \quad + \text{swap page out} \\ & \quad + \text{swap page in} \\ & \quad + \text{restart process overhead}) \end{aligned}$$

Demand Paging Example

- ❖ Memory access time = 200 nanoseconds
- ❖ Average page-fault service time = 8 milliseconds
- ❖ $EAT = (1 - p) \times (200 \text{ nanoseconds}) + p (8 \text{ milliseconds})$
 $= (1 - p) \times 200 + p \times 8,000,000$
 $= 200 + p \times 7,999,800$
- ❖ If one access out of 1,000 causes a page fault, then
 $EAT = 8,199.8 \text{ nanoseconds} \approx 8.2 \text{ microseconds.}$
This is a slowdown by a factor of 40!!

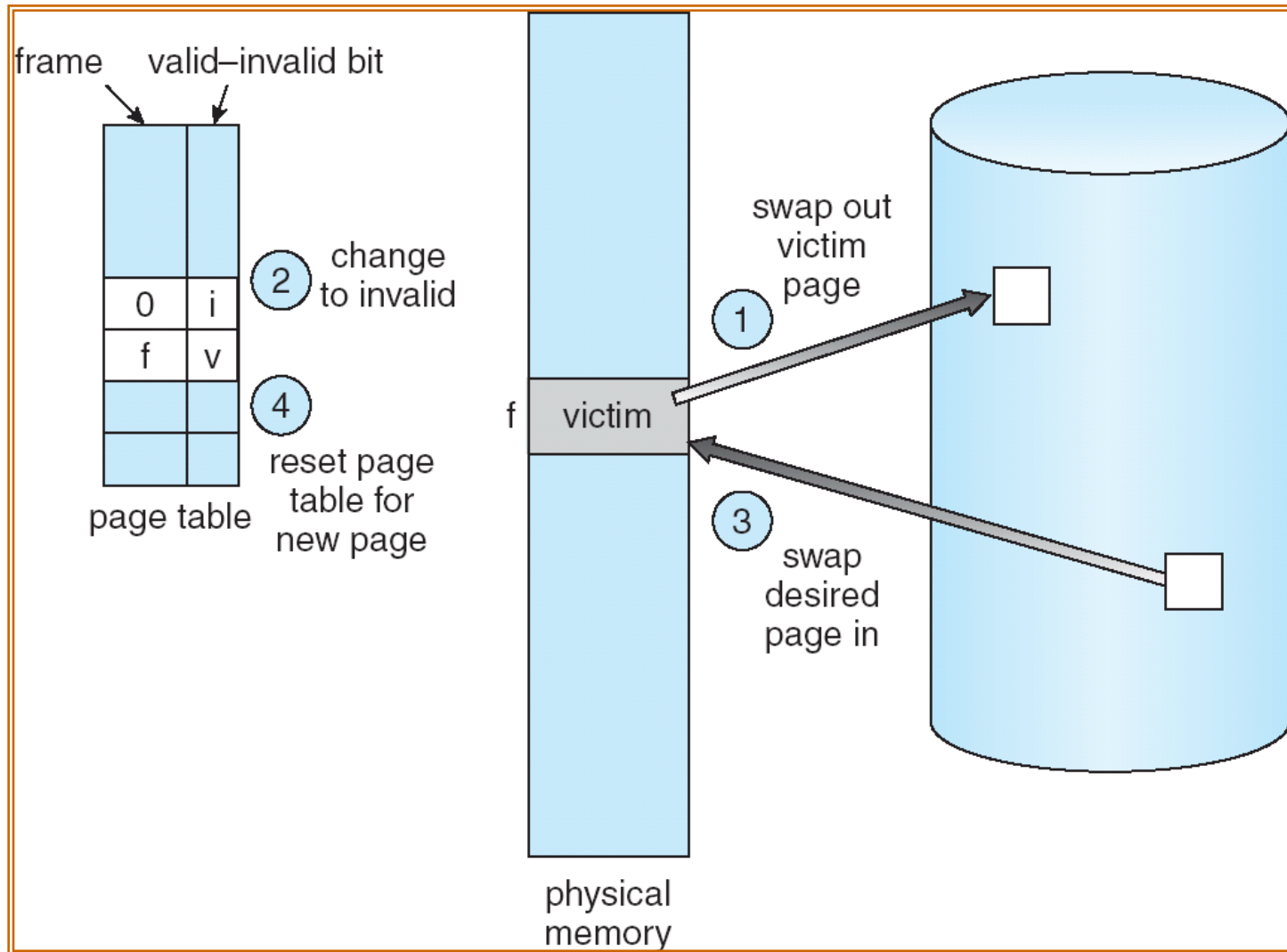
Thrashing and Prepaging

- ❖ **Thrashing:** If a process does not have “enough” pages in memory, the page-fault rate is very high and the process is busy swapping pages in and out.
- ❖ **Prepaging:** To reduce the large number of page faults that occurs at process startup, **prepage** all or some of the pages that a process will need, before they are referenced.
 - if prepaged pages are unused, I/O and memory are wasted.
 - Assume **s pages** are prepaged and **α (*fraction*)** of the pages is **used**
 - Is cost of **$s * \alpha$** save pages faults **>** or **<** than the cost of prepaging **$s * (1 - \alpha)$** unnecessary pages?
 - **α** near zero $\Rightarrow$ prepaging loses

Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use a **page replacement** algorithm to select a **victim** frame
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Restart the process

Page Replacement



Page Replacement Algorithms

- ❖ First-In First-out (**FIFO**) Algorithm
- ❖ Optimal Algorithm
- ❖ Least Recently Used (**LRU**) Algorithm
- ❖ Counting Algorithms:
 - Least Frequently Used (**LFU**) Algorithm
 - Most Frequently Used (**MFU**) Algorithm

Page Replacement Algorithms

- ❖ Want **lowest page-fault rate**
- ❖ **Evaluate algorithm** by running it on a particular **string of memory references (reference string)** and **computing the number of page faults** on that string
- ❖ In all our examples, the reference string is

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

First-In-First-Out (FIFO) Algorithm

- ❖ Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- ❖ 3 frames (3 pages can be in memory at a time per process)

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5			5	5	
	2	2	2	1	1	1			3	3	
		3	3	3	2	2			2	4	

9 page faults

- ❖ 4 frames

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1			5	5	5	5	4	4
	2	2	2			2	1	1	1	1	5
		3	3			3	3	2	2	2	2
			4			4	4	4	3	3	3

10 page faults

- ❖ **Belady's Anomaly:** more frames $\Rightarrow$ more page faults (bad choice of rep algo)

FIFO Page Replacement

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2																
	0	0	0																
		1	1																

2	2	4	4	4	0														
3	3	3	2	2	2														
1	0	0	0	3	3														

0	0																		
1	1																		
3	2																		

7	7	7																	
1	0	0																	
2	2	1																	

page frames

Optimal Algorithm

- ❖ Replace page that **will not be used** for **longest period** of time
- ❖ 4 frames example

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1			1				4	
	2	2	2			2				2	
		3	3			3				3	
			4			5				5	

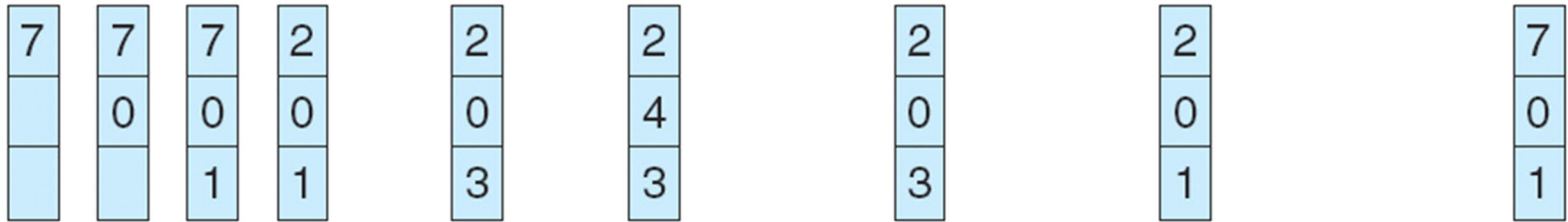
6 page faults

- ❖ How do you know this? (*Not practical – need to know future access*)
- ❖ Used for measuring how well your algorithm performs

Optimal Page Replacement

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



page frames

Least Recently Used (LRU) Algorithm

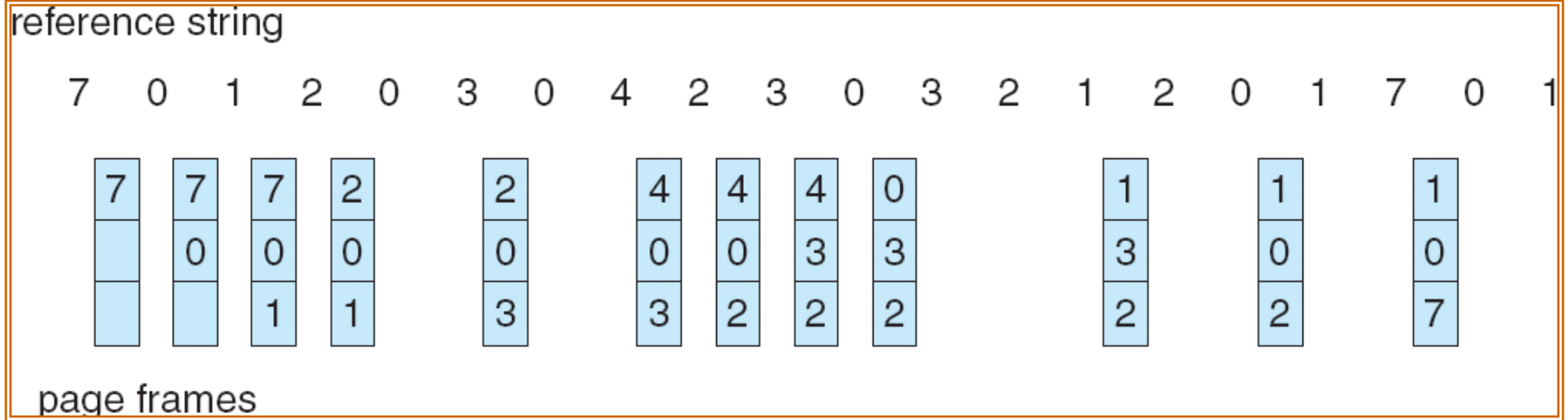
❖ Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1			1			1	1	5
	2	2	2			2			2	2	2
		3	3			5			5	4	4
			4			4			3	3	3

❖ Counter implementation

- **Every page entry has a counter**; every time page is referenced through this entry, copy the clock (current time) into the counter
- When a page needs to be changed, look at the counters to determine which are to change (Replace the frame with oldest time)

LRU Page Replacement



Counting Algorithms

- ❖ **Keep a counter** of the **number of references** that have been made to each page, since it was last loaded to the memory
- ❖ **Least Frequently Used (LFU) Algorithm:** replaces page with **smallest** count
- ❖ **Most Frequently Used (MFU) Algorithm:** based on the argument that the page with the smallest count was probably just brought in and has yet to be used

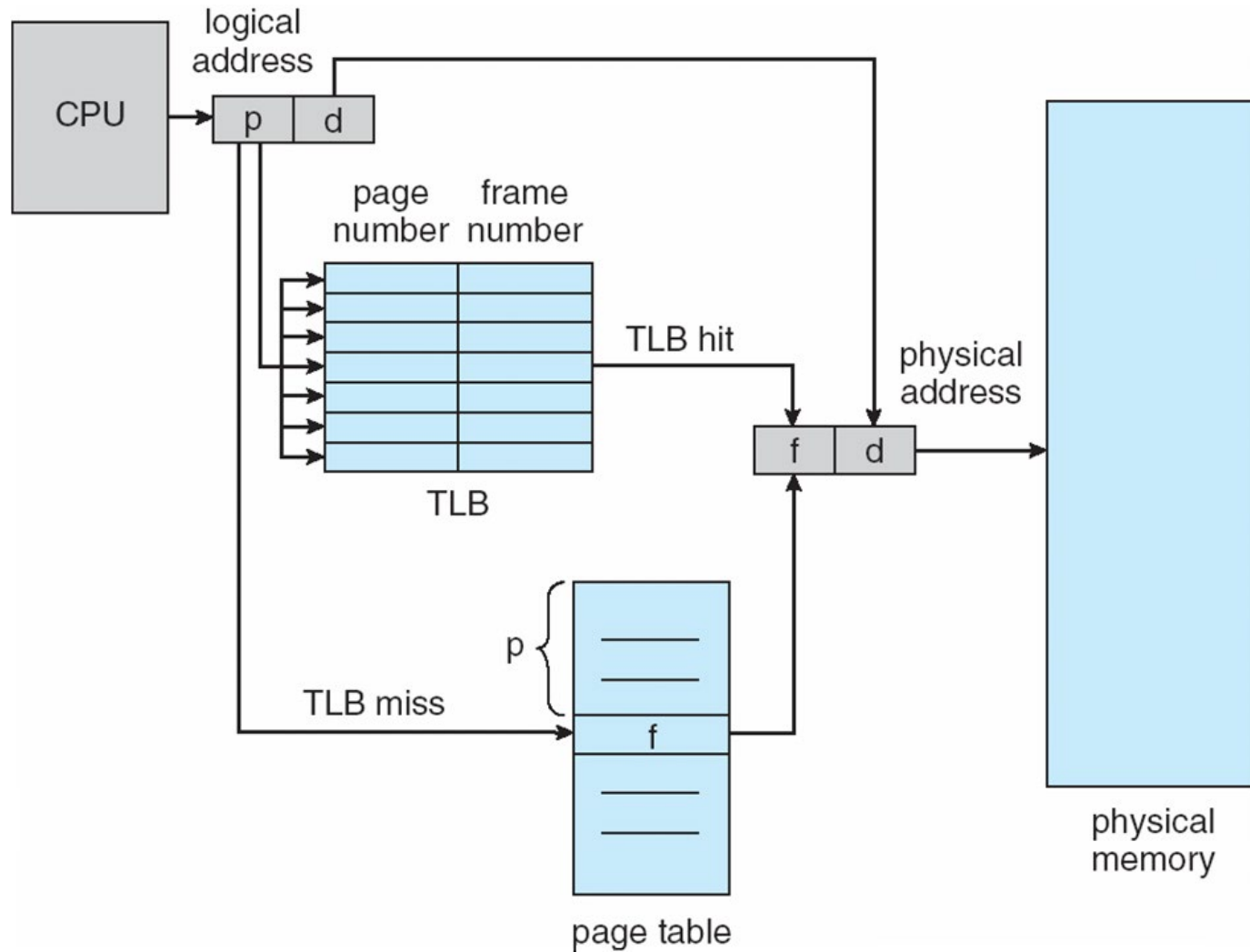
Hardware implementation of the page table

1. A set of **dedicated registers** built with **very high-speed logic**.
 - **Example:** The **address** consists of **16 bits (2^{16})**, and the **page size** is **8 KB (2^{13})**. The page table thus consists of **eight entries (2^3)** that are kept in fast registers.
 - This is **satisfactory** if the **page table** is reasonably **small** (for example, 256 entries (2^8)). But **most computers**, however, allow the **page table** to be **very large** (for example, 1 million entries (2^{20})).
2. **Two-memory accesses** implementation (page-table entry access then byte access):
 - the **page table** is kept in **main memory**, and a **page-table base register (PTBR)** points to the page table.

Hardware implementation of the page table

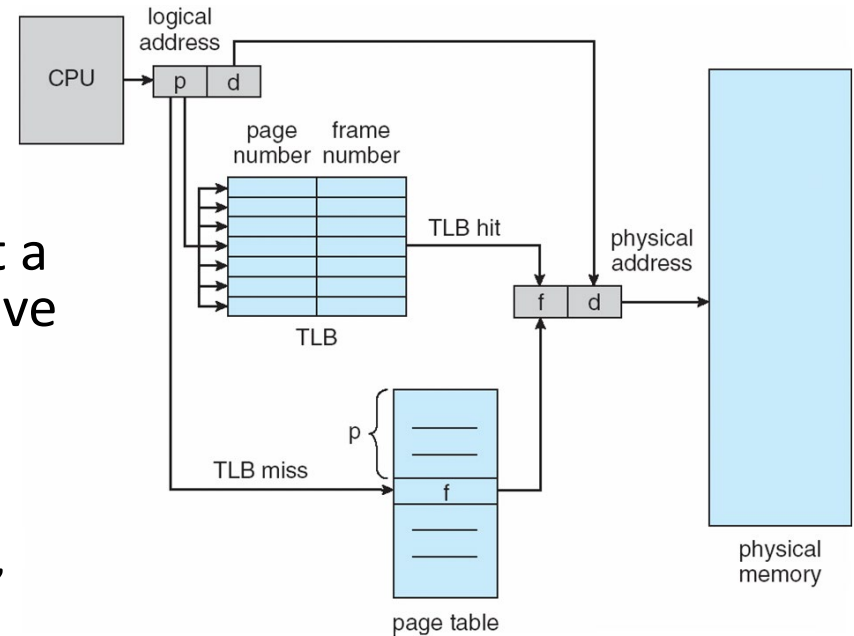
3. **Fast lookup hardware cache**, called a **Translation Look-aside Buffer (TLB)**. The **TLB** is **associative, high-speed** memory. Each **entry** in the TLB consists of **two parts**: a **key** (or **tag**) and a **value**.
 - The **TLB** contains only **a few** of the **page-table entries**. When a **logical address** is **generated** by the CPU, its **page number** is presented to the **TLB** and is **compared** with all **keys (page numbers)** **simultaneously**.
 - If it is **found** (**TLB hit**), the **corresponding value field (frame)** is returned.
 - If the **page number** is **not** in the **TLB** (known as a **TLB miss**), a **memory reference** to the **page table** must be made.
 - When the **frame number** is obtained, we can **use it to access memory**. In addition, we **add** the **page number** and **frame number** to the **TLB**, so that they will be **found quickly** on the **next reference**.
 - If the TLB is already full of entries, the operating system must select one for replacement.
 - **The search is fast**; the **hardware**, however, is **expensive**. Typically, the **number of entries** in a **TLB** is **small**, often between 64 and 1,024.

TLB Effective Memory Access Time



TLB Effective Memory Access Time

- ❖ **Associative Lookup** = ϵ time unit.
 - Time needed to search the TLB.
- ❖ Assume **memory cycle** = m time unit.
 - Time to access memory.
- ❖ **Hit ratio** α = percentage of times that a page number is found in the associative registers; ratio related to number of associative registers.
- ❖ Ratio of successful search of TLB (i.e., match page)



❖ Effective Access Time (EAT)

$$\text{EAT} = (m + \epsilon) \alpha + (2m + \epsilon)(1 - \alpha)$$

Assume that $m = 1$ microsecond then,

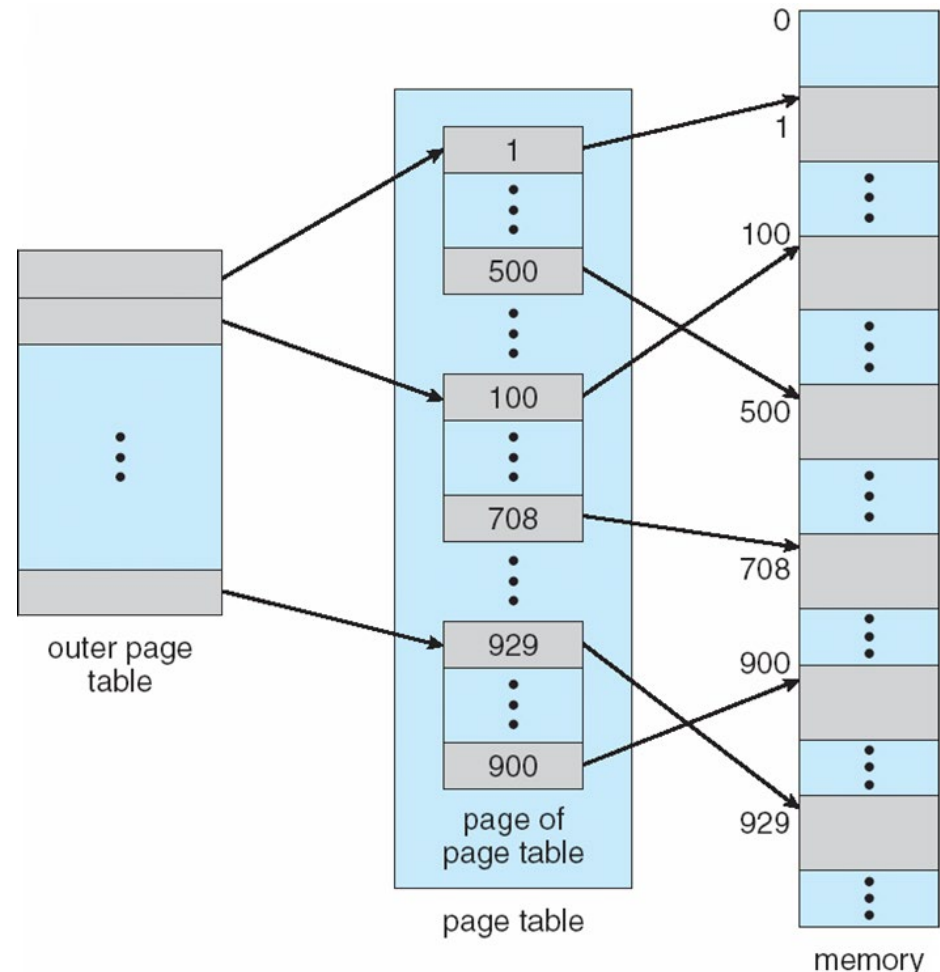
$$\text{EAT} = 2 + \epsilon - \alpha$$

Structure of the Page Table

- ❖ Hierarchical Paging
- ❖ Hashed Page Tables
- ❖ Inverted Page Tables

Hierarchical Page Tables

- ❖ In large address spaces (like 32-bit or 64-bit systems), a **single-level paging structure can become inefficient** because the page table itself can **grow very large**.
- ❖ **To manage this**, systems often use **multi-level paging**, with **two-level paging** being a common, simple structure.

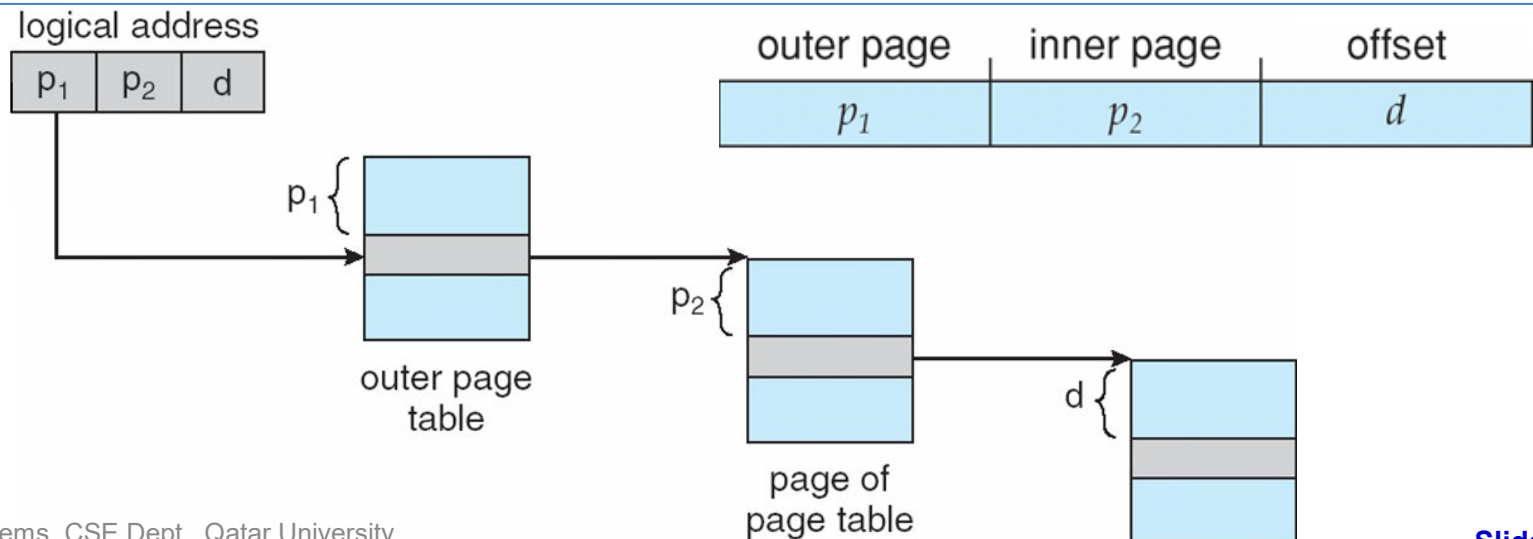


Two-Level Paging Example

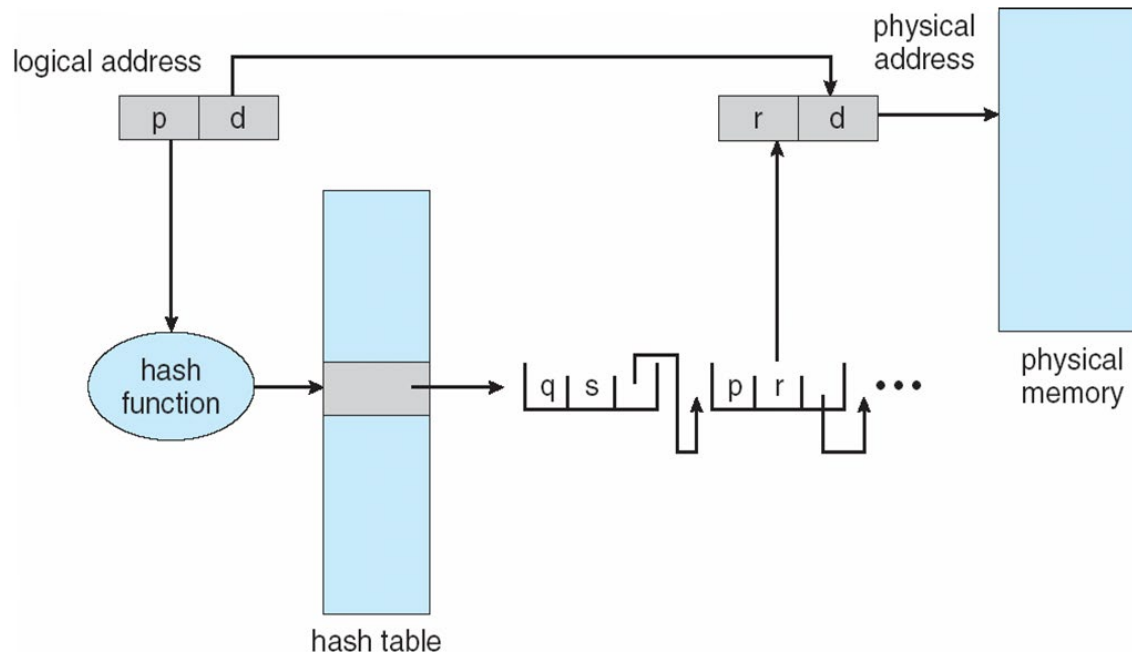
- ❖ A logical address (on **32-bit** machine with **4K page size**) is divided into:
 - a **page number** consisting of **20 bits**
 - a **page offset** consisting of **12 bits**
- ❖ Since the **page table** is **paged** and each **page table entry** has **4bytes**, then the **page number** is further divided into:
 - **10-bit** for the **outer page table index**
 - **10-bit** for the **inner page table index**
- ❖ Thus, a logical address is as follows:

page number		page offset
p_1	p_2	d
10	10	12

where p_1 is an **index** into the **outer page table**, and p_2 is the **index** into the page of the **inner page table**.

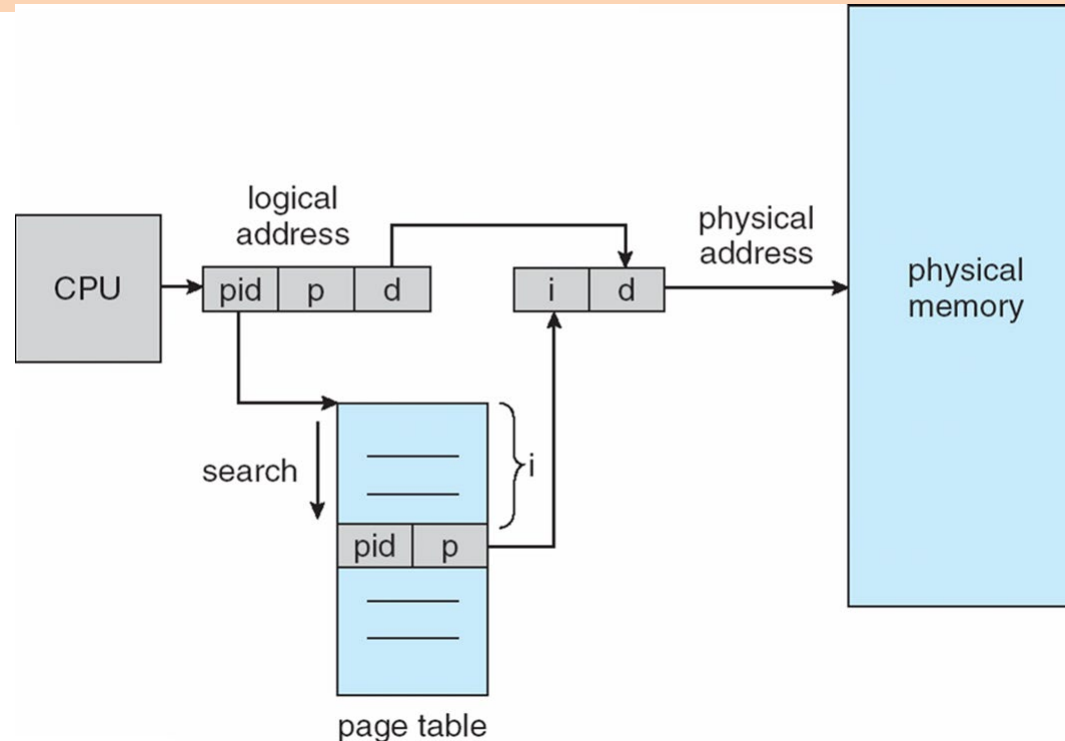


Hashed Page Tables



- ❖ Common in address spaces > 32 bits
- ❖ The **virtual page number** is **hashed** into a **hash table**
 - Each entry in the **hash table** has a **linked list** of elements hashing to the **same location** (to avoid collisions – as we can get the same value of a hash function for different page numbers)
- ❖ **Virtual page numbers** are **compared** in this **linked list** searching for a **match**
 - If a **match** is **found**, the corresponding **physical frame** is **extracted**.

Inverted Page Table



- ❖ **One entry for each real page (frame) of memory**
- ❖ **Entry** consists of the **virtual address** of the **page** stored in that **real memory location**, with information about the **process** that owns that page
- ❖ **Decreases memory** needed to **store each page table**, but **increases time** needed to **search the table** when a page reference occurs
- ❖ **Use hash table** to **limit the search to one — or at most a few — page-table entries**

LRU Implementations

❖ Counter implementation:

- Associate with each page-table entry a **time-of-use** field.
- Add to the **CPU** a **register** for a **logical clock** or **counter**.
- When a **page** is **referenced**, **copy** the **value of the clock register** to the **time-of-use field** of the **referenced page** in the page-table.
- **Replace** the **page** with **smallest time-of-use** field.
- **Drawbacks:**
 - Requires a search for replacement
 - Possible overflow of the clock register

❖ Stack implementation:

- Keep a **stack** of **page numbers**.
- If a **page** is **referenced**, **remove** it from the **stack** and put it on the **top**.
- **Always** the **most recently** used is at the **top**, and the **least recently used** is at the **bottom**. The **LRU page** is the **bottom page** pointed by the **tail** pointer.
- **Implemented** by **doubly linked list** with **head** and **tail pointers**.
- **Most appropriate** for **software** or **microcode** implementations of LRU.
- **Drawbacks:**
 - Update of the stack is expensive but the implementation requires no search for replacement.

LRU Approximations

- ❖ These **approximations** take advantage of the **hardware** maintained **bits: reference bit** and **modify/dirty bit** entries of the page table.
 - **Reference bit**: initially **0** and is **set to 1** by hardware whenever there is a **reference** to **any byte** of the **page** either **read** or **write**.
 - **Modified/dirty bit**: initially **0** and is **set to 1** by hardware when there is a **change** in the **page content** since it was brought to memory.

- ❖ These approximations are:
 - Additional-Referenced-Bits Algorithm
 - Second-Chance Algorithm
 - Enhanced Second-Chance Algorithm

LRU Approximations: Additional-Reference-Bits Algorithm

❖ The algorithm:

- keep an **8-bit byte** (shift register Initially 0s) for **each page** in a **table** in memory.
- At **regular intervals** (say, every 100 milliseconds), **shift** the **reference bit** for each **page** into the **high-order bit** of its **8-bit byte**, **shifting** the **other bits right by 1 bit** and **discarding** the **low-order bit**.
- These **8-bit shift registers** contain the **history** of **page use** for the last **eight time periods**.
- If we **interpret** these **8-bit bytes** as **unsigned integers**, the **page** with the **lowest number** is the **LRU page**, and it can be **replaced**.

❖ If the **shift register** of a page is

- **00000000**, then the page has **not been used** for **eight time periods**;
- **11111111**, then the page has been **used** at **least once** in **each period**

❖ A **page** with a **history register value** of **11000100** has been **used more recently** than one with a value of **01110111**.

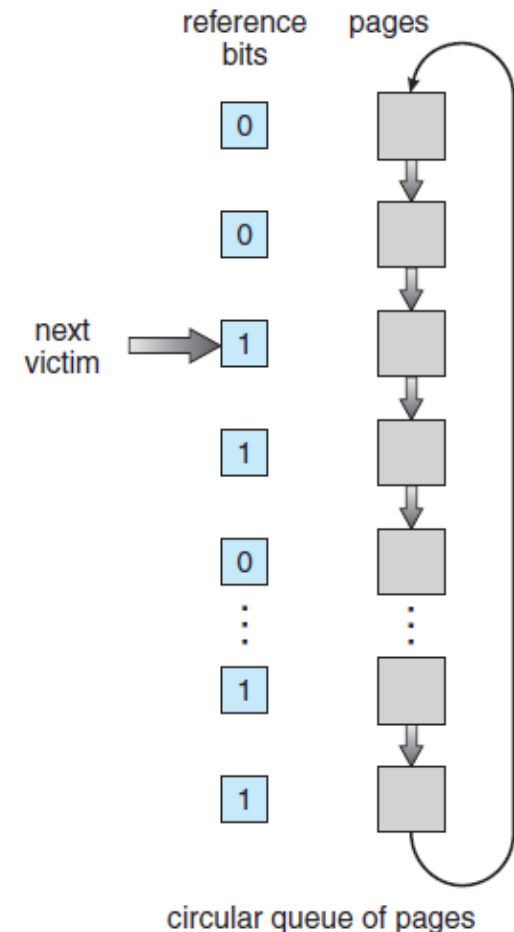
❖ In the case of **multiple pages** having the **lowest number**:

- either replace (**swap out**) **all pages** with the **smallest** value or
- use the **FIFO method** to choose among them.

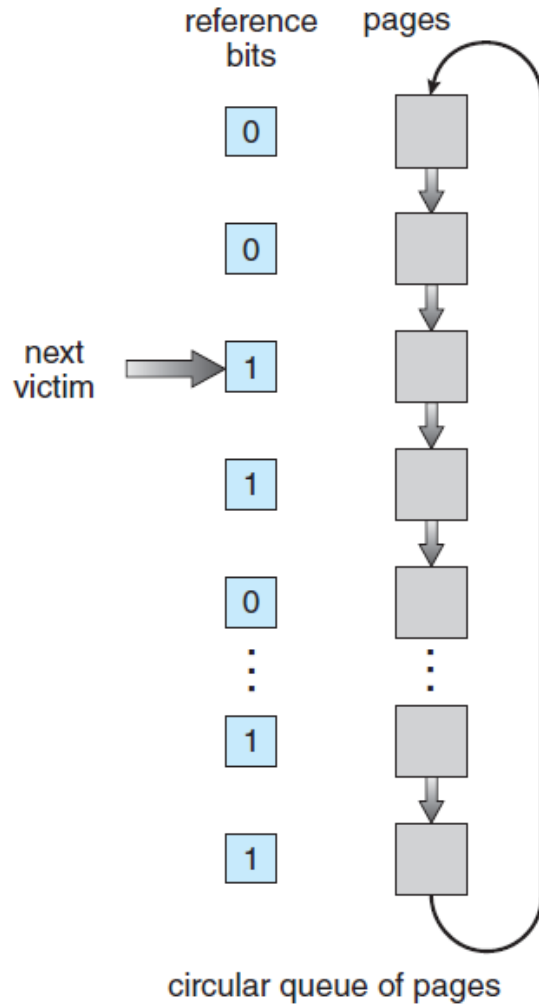
LRU Approximations: Second-Chance Algorithm

❖ The algorithm (Clock Algorithm):

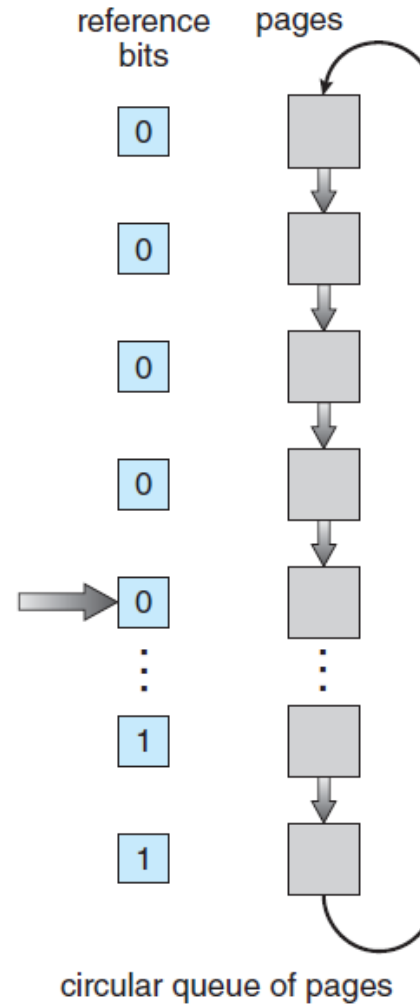
- The **basic algorithm** is a **FIFO** replacement however, when a page is selected then if the **reference bit** is
 - **0**: the **page** is **replaced**
 - **1**: the **page** gets a **second chance**, its **reference bit** is **cleared** and its **arrival time** is set to **current time**. The algorithm continues to **check the reference bit** of **next page**.
- Can be implemented as a **circular queue**.
 - When **all reference bits** are set to **1**, the algorithm **cycles the whole queue** giving each page a **second chance**.
- If a **page** that is **used often** enough to keep its **reference bit set**, it will never be replaced.



LRU Approximations: Second-Chance Algorithm



(a)



(b)

LRU Approximations: Enhanced Second-Chance Algorithm

- ❖ Considering the **reference bit** and the **modify bit** as an **ordered pair**, we have the following four possible classes:
 1. **(0, 0)** neither recently used nor modified—best page to replace
 2. **(0, 1)** not recently used but modified—not quite as good, because the page will need to be written out before replacement
 3. **(1, 0)** recently used but clean—probably will be used again soon
 4. **(1, 1)** recently used and modified—probably will be used again soon, and the page will need to be written out to disk before it can be replaced.

Each page is in one of these **four** classes.

- ❖ **The algorithm (Enhanced Clock Algorithm):**

We replace the first page encountered in the lowest nonempty class.

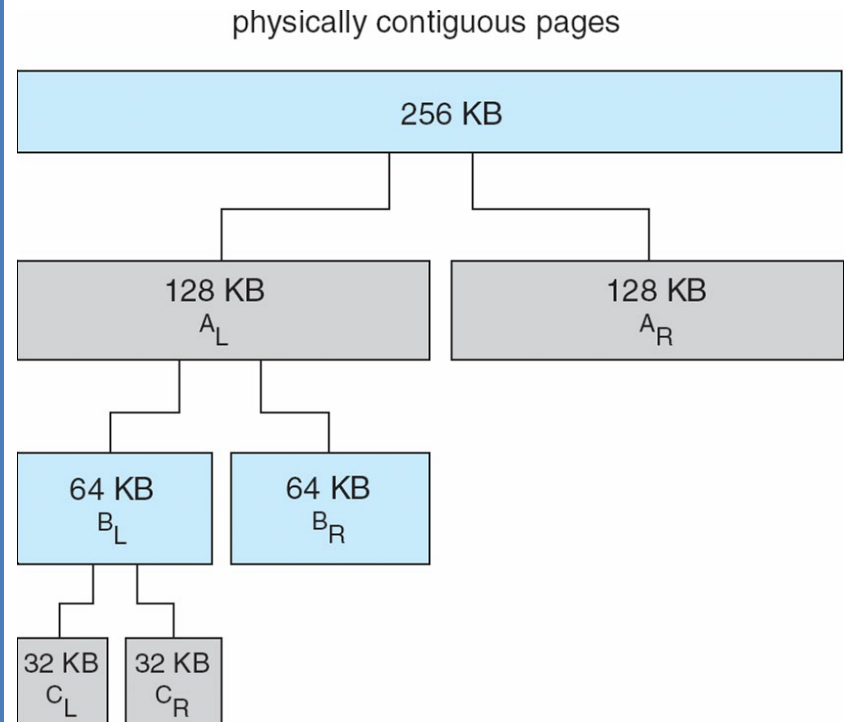
1. Cycle through the buffer looking for (0,0). If one is found, use that page.
2. Cycle through the buffer looking for (0,1). Set the **reference bit** to **zero** for all frames bypassed.
3. If step 2 failed, all **reference bits** will now be **zero** and **repetition of steps 1 and 2** are guaranteed to find a frame for replacement.

Allocating Kernel Memory

- ❖ Treated **differently** from user memory
- ❖ Often **allocated** from a **free-memory pool different** from the **list** used to satisfy **ordinary user-mode** processes.
 - Kernel requests **memory** for **structures** of **varying sizes**
 - **Some kernel memory** needs to be **contiguous**, as some **hardware devices** interact directly with physical memory – without benefit of a virtual memory interface.

Buddy System

- ❖ **Allocates memory from fixed-size segment** consisting of **physically-contiguous pages**.
- ❖ Memory allocated using **power-of-2 allocator**
 - Satisfies requests in units sized as power of 2
 - Request rounded up to nearest power of 2
 - When smaller allocation needed than is available, current chunk split into two buddies of next-lower power of 2
 - Continue until appropriate sized chunk available
- ❖ **Example:**
 - Assume **memory** of **256KB** and **kernel requests 21KB**.
 - Segment is initially divided into **two Buddies** (A_L and A_R) each **128KB**.
 - Then, **further divided** into B_L and B_R each **64KB**.
 - Then, **further divided** into C_L and C_R each **32KB**. **One of these buddies** is used to **satisfy** the **21KB** request.



- Requests can arrive in any sequence
- Memory blocks size are powers of two: $2^0, 2^1, 2^2, 2^3, \dots$
- Uses explicit tracking structures (to track free blocks)
- Allocation approximates **best-fit** behavior
- Fragmentation can still occur (internal + external)
- Blocks can be merged when freed

Buddy System

Allocating a block of size S

- ❖ Round up S to next power of 2, say 2^i (possibility of internal fragmentation)
- ❖ Access head of list for 2^i blocks
- ❖ If list isn't empty get first element.
- ❖ If list is empty (recursively) do:
 - Access head of list for the 2^{i+1} blocks
 - If list isn't empty get first element, cut element in halves, take lower half and insert upper half into list for 2^i blocks.

Releasing a block of size 2^i

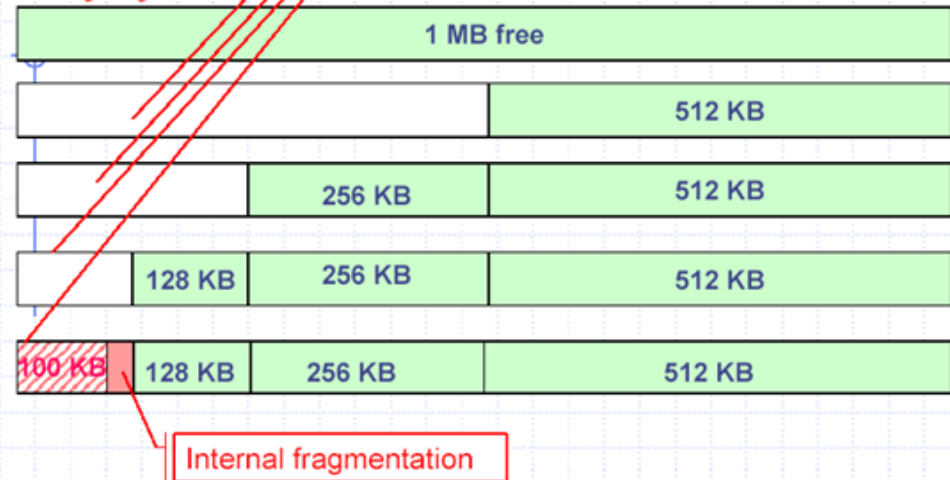
- ❖ Determine its buddy (recursively)
- ❖ If buddy is (partly) allocated, then this block to be released into the list for blocks of size 2^i
- ❖ If buddy is free, reunify both buddies in one block and insert the unified block in the list for blocks of size 2^{i+1}

Allocate memory for 100KB

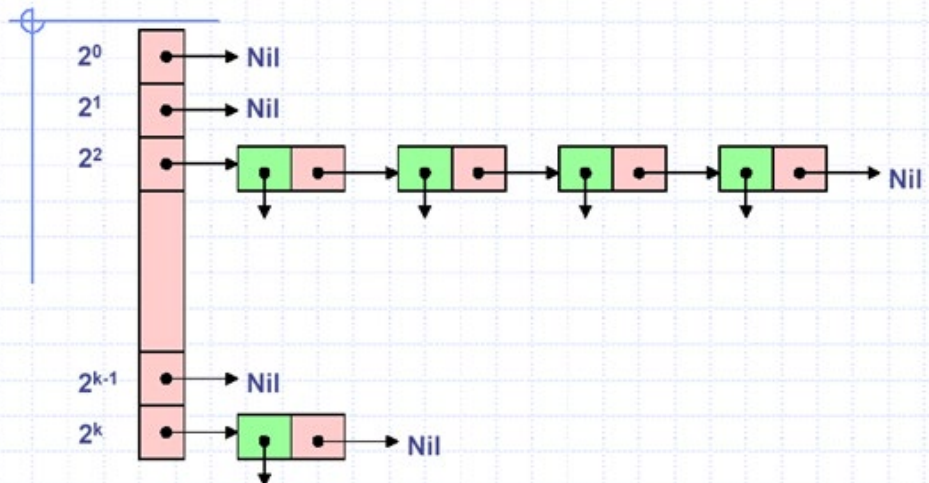
Memory Management

Buddy System

Dividing free memory until an appropriate free portion



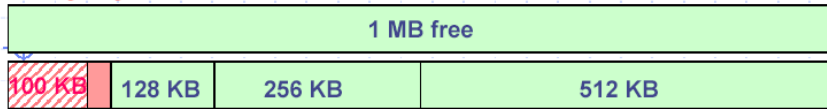
Data Structures for Buddy System



Array of heads pointing to free blocks of a certain size

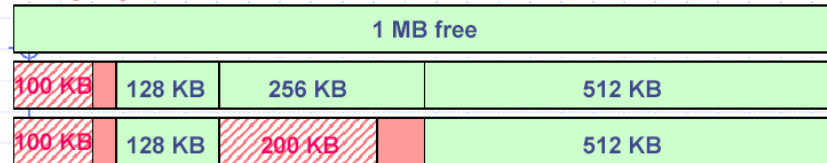
Buddy System

Buddy System



Suppose another 200 KB block is requested

Buddy System



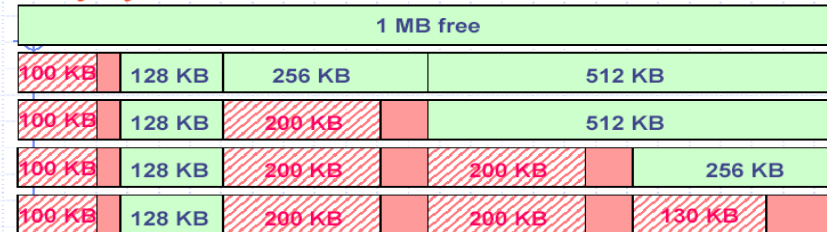
Suppose another 200 KB block is requested

Buddy System



Suppose another 130 KB block is requested

Buddy System



Suppose we have to release the first 200 KB



Buddy System



Suppose we have to release the other 200 KB



Buddy System



Can we reunify these two blocks forming a 512 KB free block?

Not at all, they are no buddies! Only buddies belong to each other!